

Funkidz Animation

Rapport technique Étape 1 et Étape 2

Correction du bug booking_time dans l'administration
Disponibilités et créneaux par animateur

Projet	Funkidz Animation - application Django
Branche Git	nassim
Date du rapport	17 septembre 2026
Serveur de test	Django runserver, port 8010
Commits	1d7baa3 (Étape 1) et eadb88e (Étape 2)
Suite de tests	33 tests - tous au vert

Workflow suivi

Les deux étapes ont été traitées avec exactement la même séquence, chaque phase devant être réellement terminée avant de passer à la suivante.

1. ANALYSE

Lecture du code existant, identification des modèles, vues, formulaires et serializers concernés. Reproduction du problème avant toute modification.

2. TRAVAIL

Modification minimale et ciblée. Aucune réécriture d'application, aucun refactoring général, aucune suppression de code fonctionnel.

3. TESTS

Tests automatisés (manage.py test) et tests réels via HTTP sur le serveur de développement réellement lancé.

4. VÉRIFICATION

Contrôle des migrations, du système Django (manage.py check), des logs serveur et de la non-régression des fonctionnalités existantes.

5. RAPPORT

Rédaction du compte rendu : cause, correction, fichiers, tests, ports.

6. COMMIT GIT

git status, git diff, ajout des seuls fichiers de l'étape, commit, vérification du hash.

Ports et accès utilisés

Le projet est un monolithe Django : il n'y a pas de frontend séparé à lancer. Les pages sont rendues par des templates Django accompagnés d'Alpine.js, servis par le même processus que l'API REST.

Élément	Valeur
Commande backend	venv/bin/python manage.py runserver 8010 --noreload
Port réellement utilisé	8010
Pourquoi pas 8000	Le port Django par défaut 8000 est déjà occupé par un autre service sur la machine (Error: That port is already in use). Aucune configuration du projet n'a été modifiée : le port est un simple argument de lancement.
URL de base	http://127.0.0.1:8010
Frontend	Aucun serveur séparé (templates Django + Alpine.js)
Compte de test	admin@funkidz.fr (connexion via /auth/login/)

Pages et endpoints réellement testés

URL	Étape	Usage
/admin/login/	1	Connexion à l'administration
/admin/bookings/booking/	1	Liste des réservations, GET et POST du tableau éditables
/admin/bookings/booking/10/change/	1	Formulaire complet, test du verrou de créneau
/auth/login/	2	Connexion côté site
/reservation/	2	Wizard de réservation en 5 étapes

URL	Étape	Usage
/api/services/	2	Liste des prestations
/api/bookings/	2	Création réelle d'une réservation (POST)
/api/availability/check/	2	Vérification d'un créneau, avec et sans animateur
/api/availability/slots/	2	Créneaux du jour, avec et sans animateur
/api/availability/animators/	2	Nouveau : calendrier par animateur

Étape 1 - Correction du bug booking_time

Le problème signalé

Dans l'administration Django, modifier le statut d'une ou plusieurs réservations directement dans le tableau puis cliquer sur Enregistrer provoquait une erreur 500. Le client l'avait rapportée sous la forme : `FieldError: Booking has no field named 'booking_time'.`

Ce que la reproduction a réellement montré

Le bug a d'abord été reproduit sur le projet avant toute modification. L'erreur exacte est :

```
ValueError: 'BookingForm' has no field named 'booking_time'.
```

Le champ `booking_time` existe bien dans le modèle `Booking` et en base de données. Le problème n'était donc pas un champ manquant, mais une erreur de validation rattachée à un champ absent du formulaire.

Cause exacte

- `BookingAdmin` déclare `list_editable = ('status', 'final_price')` : le formulaire généré pour chaque ligne du tableau ne contient que ces deux champs.
- À la validation, Django appelle `Booking.full_clean()`, donc `Booking.clean()`.
- `Booking.clean()` rejouait le contrôle de disponibilité du créneau sur toutes les lignes au statut `PENDING` ou `CONFIRMED`, y compris sur des créneaux enregistrés depuis longtemps.
- En cas d'indisponibilité, `clean()` levait `ValidationError({'booking_time': ...})`.
- Django tente alors de rattacher cette erreur au champ `booking_time`, qui n'existe pas dans le formulaire de la liste éditée. D'où le `ValueError` et l'erreur 500.

Pourquoi le bug semblait aléatoire

Il suffisait qu'une ligne affichée dans le tableau, même non modifiée, soit en collision de créneau. La ligne que l'on annulait n'était pas forcément la cause : toutes les lignes de la page sont validées au moment de l'enregistrement.

Correction appliquée

Le contrôle de disponibilité ne s'exécute désormais que si la date, l'heure ou la prestation ont réellement changé. Une simple modification de statut ou de prix sur une réservation existante ne peut donc plus échouer sur la disponibilité du créneau.

```
def _slot_has_changed(self):
    if not self.pk:
        return True
    previous = Booking.objects.filter(pk=self.pk).values(
        'booking_date', 'booking_time', 'service_id').first()
    ...

def clean(self):
    ...
    if not self._slot_has_changed():
        return # rien à revalider : le créneau n'a pas bougé
```

Le verrouillage de créneau reste entièrement actif à la création d'une réservation et sur le formulaire complet, ou le

champ booking_time est présent.

Fichiers modifiés

Fichier	Raison
bookings/models.py	Ajout de Booking._slot_has_changed() et garde en début de Booking.clean(). 22 lignes ajoutées, aucune ligne supprimée.

Aucun autre fichier touché : ni Stripe, ni les emails, ni les rôles, ni le frontend.

Migrations

Aucune migration nécessaire

La correction est purement applicative : aucun champ ajouté, modifié ou supprimé.

Vérification : `manage.py makemigrations --check --dry-run -> No changes detected`

Résultats des tests

#	Test	Résultat obtenu	Verdict
1	Affichage de la liste des réservations	HTTP 200, 5 lignes, aucune erreur booking_time	OK
2	Changement de statut d'une seule ligne	HTTP 302, statut enregistré	OK
3	Changement de statut de plusieurs lignes	HTTP 302, statuts enregistrés	OK
4	Intégrité des données (date, heure, prix, prestation, client, adresse)	Snapshot avant / après identique	OK
5	Logs serveur pendant les tests	23 requêtes, 0 réponse 4xx ou 5xx, 0 traceback	OK
6	Scénario exact du bug : collision de créneau dans le tableau puis annulation	HTTP 302. Avant correction : ValueError et erreur 500 (reproduit)	OK
7	Non-régression du verrou : déplacer une réservation sur un créneau occupé	HTTP 200, message Ce créneau est indisponible affiché, base inchangée	OK
8	manage.py test	Ran 23 tests - OK	OK
9	manage.py check	System check identified no issues	OK

Commit

Élément	Valeur
Hash	1d7baa3 (1d7baa3801b72f741c9e16c0df2201fe957b943e)
Branche	nassim
Message	fix: correction booking_time dans admin reservations
Contenu	1 fichier, 22 insertions, 0 suppression

Étape 2 - Créneaux et disponibilités par animateur

Le besoin

Retour transmis par le professeur : chaque animateur doit avoir son propre calendrier. Une réservation doit bloquer uniquement l'animateur concerné, et non l'ensemble de l'équipe.

Situation	Comportement attendu
Même date, même heure, animateurs différents	Autorisé : les réservations coexistent si les animateurs sont disponibles
Même date, même heure, même animateur	Refusé : message Ce créneau est indisponible

Ancienne logique et pourquoi elle bloquait tout le monde

Dans `availability/views.py`, la première étape de `is_slot_available_for_booking` verrouillait globalement :

```
existing_overlapping = Booking.objects.filter(
    booking_date=booking_date,
    status__in=[CONFIRMED, PENDING])
for b in existing_overlapping:
    if chevauchement:
        return False, "Ce créneau est indisponible."
```

Ce filtre ne regardait ni l'animateur, ni la table `BookingAssignment`. N'importe quelle réservation active chevauchant l'horaire fermait le créneau pour tout le monde. L'étape suivante parcourait bien les animateurs un par un, mais elle n'était atteinte que si l'étape 1 n'avait rien bloqué. En pratique, une seule réservation à 14h00 rendait 14h00 indisponible pour les six animateurs.

Nouvelle logique

Mode animateur précis (paramètre `animateur_id`)

Le calendrier individuel fait foi : même animateur, même date, créneau chevauchant égale conflit. Un animateur est indisponible s'il est en congé validé, s'il a bloqué la plage horaire, ou s'il a déjà une mission attribuée qui chevauche le créneau.

Mode pool (parcours client, aucun animateur choisi)

- On calcule la liste des animateurs réellement libres sur ce créneau.
- On soustrait les réservations actives déjà posées sur ce créneau et pas encore attribuées : chacune mobilisera un animateur.
- Le créneau est disponible s'il reste au moins un animateur.

L'anti-doublon n'a pas disparu

Il est passé d'un verrou global à une contrainte de capacité réelle.
Avec 6 animateurs, 5 réservations peuvent coexister à 14h00 ; la 7e est refusée.
Les messages utilisateur existants sont conservés.

Fichiers modifiés

Fichier	Raison
availability/views.py	Coeur de la correction : nouveaux helpers <code>is_animator_free()</code> et <code>get_animators_availability()</code> , calcul de la capacité du pool, paramètre animateur optionnel sur <code>/check/</code> et <code>/slots/</code> , compteur d'animateurs libres, nouvel endpoint <code>/api/availability/animators/</code> .
availability/tests.py	Ajout de 10 tests couvrant les scénarios par animateur. Aucun test existant modifié.
templates/booking/wizard.html	4 lignes ajoutées : affichage du nombre d'animateurs disponibles sous chaque créneau. Aucun changement de design ni de parcours.

Non modifiés : Stripe, emails, rôles utilisateurs, administration, modèles, migrations, `bookings/serializers.py` et `bookings/models.py`.

Migrations

Aucune migration nécessaire

La relation réservation / animateur existait déjà via `BookingAssignment`, ainsi que `Availability`, `WeeklySchedule` et `AnimateurLeave`, tous rattachés à `AnimateurProfile`. La correction est purement applicative.

Vérification : `makemigrations --check --dry-run` -> No changes detected.

Aucune donnée supprimée : les 5 réservations existantes sont intactes.

Aucune contrainte d'unicité ajoutée : le conflit porte sur un chevauchement d'intervalles horaires, dont la durée varie selon la prestation. Une contrainte SQL `UniqueConstraint` ne sait pas exprimer cela. L'anti-doublon reste applicatif, au même endroit qu'avant.

Résultats des tests automatisés

Contexte : 3 animateurs A, B et C, prestation de 2 heures, date X = 2026-06-15, heure Y = 14:00.

#	Scénario	Résultat obtenu	Verdict
1	Réservation à 14:00 affectée à A, puis interroger A	Indisponible : Déjà en mission de 14:00 à 16:00	OK
2	Interroger B et C sur le même créneau	Disponibles. 2 animateurs libres. 2e réservation acceptée	OK
3	Affecter A à une 2e réservation sur le même créneau	Refusé : <code>ValidationError</code> levée par <code>BookingAssignment.clean()</code>	OK
4	A, date X, 17:00 (pas de chevauchement)	Disponible	OK
5	A, date X + 1 jour, 14:00	Disponible	OK
6	Pool saturé : les 3 animateurs affectés à 14:00	Indisponible	OK
7	Pool : 3 réservations non attribuées à 14:00	Indisponible : tous nos animateurs sont déjà mobilisés	OK
8	<code>/api/availability/check/</code> avec paramètre animateur	A refusé, B accepté, <code>animators_available = 2</code>	OK
9	<code>/api/availability/animators/</code>	3 animateurs, A indisponible, B et C disponibles	OK
10	<code>/api/availability/slots/</code> avec et sans animateur	14:00 ouvert pour le pool, fermé pour A	OK

Suite complète

manage.py test -> Ran 33 tests - OK (23 avant l'étape, 10 ajoutés).

Les 9 tests de disponibilité préexistants passent sans aucune modification.

Résultats des tests réels sur le serveur

Base de données du projet, 6 animateurs. Date X = 2026-10-27, heure Y = 14:00, animateur A = Lucas Dupont, animateur B = Sophie Martin.

#	Scénario	Résultat obtenu	Verdict
1	Réservation #11 affectée a Lucas, puis check?animateur=Lucas	HTTP 200, available=false, Déjà en mission de 14:00 à 16:00	OK
2	check?animateur=Sophie, même date et même heure	HTTP 200, available=true, 5 animateurs libres sur 6	OK
2b	POST /api/bookings/ sur le même créneau (parcours client réel)	HTTP 201, réservation #12 créée. Impossible avant la correction	OK
3	Affectation de Lucas à la 2e réservation	Refusée : L'animateur a déjà une mission de 14:00:00 à 16:00:00	OK
4	Lucas, même date, 18:00	available=true	OK
5	Lucas, 2026-10-28, 14:00	available=true	OK
6	Parcours complet : wizard, prestations, créneaux, calendrier animateur	Wizard HTTP 200 avec ses 5 étapes intactes, 10 prestations, 7 créneaux avec le compteur d'animateurs, slots?animateur=Lucas fermé à 14:00, endpoint animators 5 sur 6	OK

Logs serveur

19 requêtes, 0 traceback, 0 erreur 5xx.

Les seules réponses 404 proviennent de deux appels sur une mauvaise URL de connexion pendant la mise au point du script de test, pas de l'application.

Les réservations de test #11 et #12 ont été supprimées : la base est revenue à ses 5 réservations d'origine.

Régressions

Aucune régression constatée.

- Les 23 tests préexistants passent tous, sans avoir été modifiés.
- Le blocage reste effectif quand il doit l'être : congés validés, plages bloquées par l'animateur, pool saturé, même animateur sur un créneau chevauchant.
- Le correctif de l'étape 1 reste actif et couvert par ses propres tests.
- La signature de is_slot_available_for_booking reste rétro-compatible : le nouveau paramètre est optionnel, les appelants existants fonctionnent tels quels.

Changement de comportement volontaire

Le créneau 14:00 reste désormais ouvert tant que des animateurs sont libres. Cela annule le choix verrouillage strict acté précédemment, conformément au retour transmis par le professeur. Cette contradiction était déjà signalée dans le document retours/Retour client - 16 septembre 2026.md.

Commit

Élément	Valeur
Hash	eadb88e (eadb88ecc7a22a0ed83fabeeec7cd096d817cc0f6)
Branche	nassim
Message	feat: disponibilites et creneaux par animateur
Contenu	3 fichiers, 457 insertions, 65 suppressions

Synthèse

	Étape 1	Étape 2
Objet	Bug booking_time dans l'admin	Créneaux par animateur
Fichiers modifiés	1	3
Migration	Aucune	Aucune
Tests de l'étape	9 sur 9	10 sur 10
Suite complète	23 tests - OK	33 tests - OK
Erreurs serveur	0	0
Régression	Aucune	Aucune
Commit	1d7baa3	eadb88e

État du dépôt à la fin des deux étapes

- Branche nassim, deux commits ajoutés : 1d7baa3 puis eadb88e.
- Le fichier .gitignore reste modifié mais non commité : il ajoute retours/ et docs/ à la liste des fichiers ignorés. Cette modification existait avant le début des deux étapes et a été volontairement laissée de côté pour garder des commits propres.
- La base de développement db.sqlite3 n'est pas suivie par Git. Les données créées pendant les tests ont été supprimées et les 5 réservations d'origine restaurées.

Ce qui reste à traiter dans le retour client

- Trois adresses email distinctes (admin, client, animateur) pour la démonstration orale.
- Lisibilité du panel admin : couleur par statut et tri des réservations par date et heure.
- Paiement : formulaire de saisie de carte bancaire visible, attendu par le jury.
- Doublet Nos Formules et Tarifs & Options, à fusionner.
- Fusion avec la version parallèle de l'application, encore à identifier.
- Base de données et dictionnaire des données pour le rapport provisoire.